Supriam Das
Supriam.das@gmail.com
27 March 2026

1 Structured Data

1.1 Ex1

A Bash script was created to convert XML data into CSV format for both student and faculty files. The script takes an input XML file and an output CSV file as arguments. It first checks whether the input file contains student data by searching for the <student> tag. If it is a student file, the script writes the appropriate header and uses awk with angle brackets as delimiters to extract values from each XML tag. The values are stored in variables as the file is read line by line, and when the closing </student> tag is reached, a complete row is printed. Missing exam marks are replaced with NULL to ensure consistency. If the file does not contain student data, it is treated as a faculty file, and the corresponding fields are extracted and written to the CSV file.

```
#!/bin/bash

input=$1 output=$2 if grep -q "<student>" "$input"; then echo

"student_name,student_id,programme,year,module_id,module_name,faculty,building,room,exam_mark" > $o

awk -F"[<>]" ’
/<student_name>/ { name=$3 }
/<student_id>/ { id=$3 }
/<programme>/ { prog=$3 }
/<year>/ { year=$3 }
/<module_id>/ { mid=$3 }
/<module_name>/ { mname=$3 }
/<faculty>/ { fac=$3 }
/<building>/ { bld=$3 }
/<room>/ { room=$3 }
/<exam_mark>/ { mark=$3 }
/<\/student>/ { if (mark == "")
        mark="NULL"
        print name "," id "," prog "," year "," mid "," mname "," fac "," bld "," room "," mark mark=""
}
’ $input >> $output else

echo "faculty,building,room,capacity" > $output

awk -F"[<>]" ’
/<faculty>/ { name=$3 }
/<building>/ { building=$3 }
/<room>/ { room=$3 }
/<capacity>/ { capacity=$3 print name "," building "," room ","
        capacity
}
’ $input >> $output fi
```

2 Relational Model

2.1 Ex2

Relations

Based on the generated CSV files, the following relations are identified:

- Students (student name, student id, programme, year, module id, module name, faculty, building, room, exam mark)
- Faculties (faculty, building, room, capacity)

Students Relation

Primary Key: {student id, module_id}

student_id → student name, programme, year

module_id → module name

(room, building) → faculty

(student id, module_id) → exam mark, room, building

Justification: The relation represents student enrolments. A student can take multiple modules and each module can have multiple students, so neither attribute alone is sufficient. The combination of student id and module id uniquely identifies each record.

Faculties Relation

Primary Key: {building, room}

(building, room) → faculty, capacity

Justification: Room identifiers are not unique across the university, but the combination of building and room uniquely identifies each room and determines its faculty and capacity.

3 Normalisation

3.1 Ex3

The original student XML data was not in First Normal Form (1NF), as it contained repeating groups of module-related information within each student element. A single student could be associated with multiple modules, which violates the requirement that each record should contain only atomic values.

To convert this into 1NF, the data was restructured so that each row represents a single student–module combination. This transformation was achieved when generating the CSV file, where each module taken by a student is stored as a separate row. As a result, all attributes now contain atomic values and there are no repeating groups.

In contrast, the faculty data did not require any changes. The original faculty XML was already in 1NF, as each record represents a single room with atomic attributes and no repeating groups.

3.2 Ex4

The primary key of the Students relation is the composite key {student id, module id}. Partial dependencies occur when non-key attributes depend on only part of this composite key. The following partial dependencies are identified:

$$student_id \rightarrow student_name, programme, year$$
$$module_id \rightarrow module_name$$

These dependencies indicate that some attributes depend only on a subset of the primary key, which violates the requirements of Second Normal Form (2NF).

These violate 2NF, so the relation is decomposed into the following relations:

- Student (student id, student name, programme, year)
- Module (module _id, module name)

The remaining attributes stay in the original relation, which will later represent the enrolment relationship between students and modules.

3.3 Ex5

To convert the data into Second Normal Form (2NF), the partial dependencies identified in Ex4 were removed by decomposing the original relation into separate relations.

The resulting relations are shown below.

Student

Attribute	Description
student id	Primary key
student name	Student name
programme	Degree programme
year	Year of study

Module

Attribute	Description
module id	Primary key
module name	Module name

Enrollment

Attribute	Description
student id	References Student
module id	References Module
faculty	Faculty name
building	Building identifier
room	Room number
exam mark	Exam result

Each non-key attribute is now fully dependent on the entire primary key of its respective relation. This removes partial dependencies and ensures the data is in Second Normal Form.

3.4 Ex6

The relations obtained in Ex5 are not yet fully in Third Normal Form (3NF) for student.csv. This is due to the presence of a transitive dependency. A transitive dependency occurs when an attribute depends on another non-key attribute, such that if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

The Room relation, derived from the faculty data, is already in Third Normal Form. The attributes faculty and capacity depend directly on the combination of building and room, which forms the key. There are no transitive dependencies within this relation, so no further decomposition is required.

In the Enrollment relation, faculty depends on the combination of room and building, rather than directly on the primary key. This means that faculty is transitively dependent on the key through (room, building).

To achieve 3NF, this dependency was removed by separating the room-related information into its own relation. The process followed was to examine each relation from Ex5 and check whether any non-key attribute depended on another non-key attribute. When such a dependency was found, the attributes involved were moved into a new relation where they depend only on their own key.

The final relations in 3NF are shown below.

Student

Attribute	SQLite Datatype	Key

student id	INTEGER	Primary key
student name	TEXT	–
programme	TEXT	–
year	INTEGER	–

Module

Attribute	SQLite Datatype	Key
module id	TEXT	Primary key
module name	TEXT	–

Room

Attribute	SQLite Datatype	Key
building	TEXT	Part of primary key
room	TEXT	Part of primary key
faculty	TEXT	–
capacity	INTEGER	–

Enrollment

Attribute	SQLite Datatype	Key
student id	INTEGER	Part of primary key
module id	TEXT	Part of primary key
exam mark	INTEGER	–
building	TEXT	Foreign key
room	TEXT	Foreign key

By separating the Room relation, the transitive dependency was removed. Each non-key attribute now depends only on the primary key of its relation, so the schema satisfies Third Normal Form.

4 Modelling

4.1 Ex7

Based on the final relations obtained after normalisation to Third Normal Form, tables were designed for implementation in SQLite. Appropriate datatypes were chosen according to the type of data stored in each attribute, with INTEGER used for numeric values and TEXT used for descriptive values.

Student

Attribute	SQLite Datatype
student id	INTEGER
student name	TEXT
programme	TEXT
year	INTEGER

Module

Attribute	SQLite Datatype
module id	TEXT
module name	TEXT

Room

Attribute	SQLite Datatype
building	TEXT
room	TEXT
faculty	TEXT

capacity	INTEGER

Enrollment

Attribute	SQLite Datatype
student id	INTEGER
module id	TEXT
exam mark	INTEGER
building	TEXT
room	TEXT

4.2 Ex8

An SQLite database named university.db was created to store the data. The CSV files produced in Ex1 were then imported into the database using the SQLite .import command.

Before importing, two tables were created to store the raw data. These tables were named students raw and faculty raw to ensure they do not conflict with the normalised relations created later.

```
.mode csv

DROP TABLE IF EXISTS students_raw;
DROP TABLE IF EXISTS faculty_raw;

CREATE TABLE students_raw (
    student_name TEXT,
    student_id INTEGER,
    programme TEXT, year
    INTEGER, module_id TEXT,
    module_name TEXT, faculty
    TEXT, building TEXT, room
    TEXT, exam_mark TEXT
);

CREATE TABLE faculty_raw ( faculty TEXT,
    building TEXT, room
    TEXT, capacity
    INTEGER
);

.import students.csv students_raw
.import faculty.csv faculty_raw
```

The .mode csv command ensures that SQLite reads the input files in CSV format. Existing tables are dropped first to avoid conflicts when re-running the script. The raw tables are then created with attributes matching the structure of the CSV files. Finally, the .import command loads the data from each CSV file into its corresponding table.

4.3 Ex9

The following SQL statements were used to create and populate the normalised tables based on the imported raw data. Existing tables are dropped first to ensure the script can be executed multiple times without errors.

```
DROP TABLE IF EXISTS Enrollment;
DROP TABLE IF EXISTS Student;
DROP TABLE IF EXISTS Module;
```

```
DROP TABLE IF EXISTS Room;

CREATE TABLE Student ( student_id INTEGER
      PRIMARY KEY, student_name TEXT,
      programme TEXT, year INTEGER
);

CREATE TABLE Module ( module_id
      TEXT PRIMARY KEY, module_name
      TEXT
);

CREATE TABLE Room (
      building TEXT, room
      TEXT, faculty TEXT,
      capacity INTEGER,
      PRIMARY KEY (building, room)
);

CREATE TABLE Enrollment (
      student_id INTEGER,
      module_id TEXT,
      exam_mark INTEGER,
      building TEXT, room TEXT,
      PRIMARY KEY (student_id, module_id),
      FOREIGN KEY (student_id) REFERENCES Student(student_id),
      FOREIGN KEY (module_id) REFERENCES Module(module_id),
      FOREIGN KEY (building, room) REFERENCES Room(building, room)
);
INSERT INTO Student (student_id, student_name, programme, year)
SELECT DISTINCT student_id, student_name, programme, year
FROM students_raw
WHERE student_id != 'student_id';

INSERT INTO Module (module_id, module_name)
SELECT DISTINCT module_id, module_name
FROM students_raw
WHERE module_id != 'module_id';

INSERT INTO Room (building, room, faculty, capacity)
SELECT DISTINCT building, room, faculty, capacity
FROM faculty_raw
WHERE building != 'building';

INSERT INTO Enrollment (student_id, module_id, exam_mark, building, room)
SELECT student_id, module_id, exam_mark, building, room
FROM students_raw
WHERE student_id != 'student_id';
```

The DISTINCT keyword was used when inserting into the Student, Module and Room tables to remove duplicate entries and ensure that each entity appears only once. The Enrollment table stores relationships between students and modules, so each row represents a valid student–module combination and DISTINCT was

not required. The DROP TABLE statements ensure that the script can be re-executed without conflicts with existing data. Header rows from the CSV files were excluded using WHERE conditions. All data was taken directly from the raw tables, ensuring that the original data remained unchanged as required.

5 Querying

5.1 Ex10

The following SQL query was written to calculate the total capacity of rooms in each building:

```
SELECT building, SUM(capacity) AS total_capacity
FROM Room
GROUP BY building
ORDER BY CAST(SUBSTR(building, 2) AS INTEGER);
```

The query groups all rooms by their building and calculates the total capacity using the SUM function. The ORDER BY clause is used to sort the buildings numerically by extracting the numeric part of the building identifier, ensuring that buildings are listed in the correct order (e.g. B1, B2, ..., B100).

5.2 Ex11

The following SQL query was written to find each Year 1 Computer Science student along with their average exam mark:

```
SELECT s.student_id, s.student_name, AVG(e.exam_mark) AS avg_mark
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
WHERE s.year = 1
AND s.programme = 'Computer Science'
AND e.exam_mark IS NOT NULL
GROUP BY s.student_id, s.student_name
ORDER BY avg_mark DESC;
```

The query joins the Student and Enrollment tables to combine student information with their exam marks. It filters only Year 1 students enrolled in the Computer Science programme and excludes any missing exam marks. The AVG function is used to calculate the average exam mark for each student. The results are grouped by student and sorted in descending order so that students with the highest average marks appear first.